Exercise 10

Elisa Schöpf, Leon Krieger, Nikola Adzic

May 31, 2026

Contents

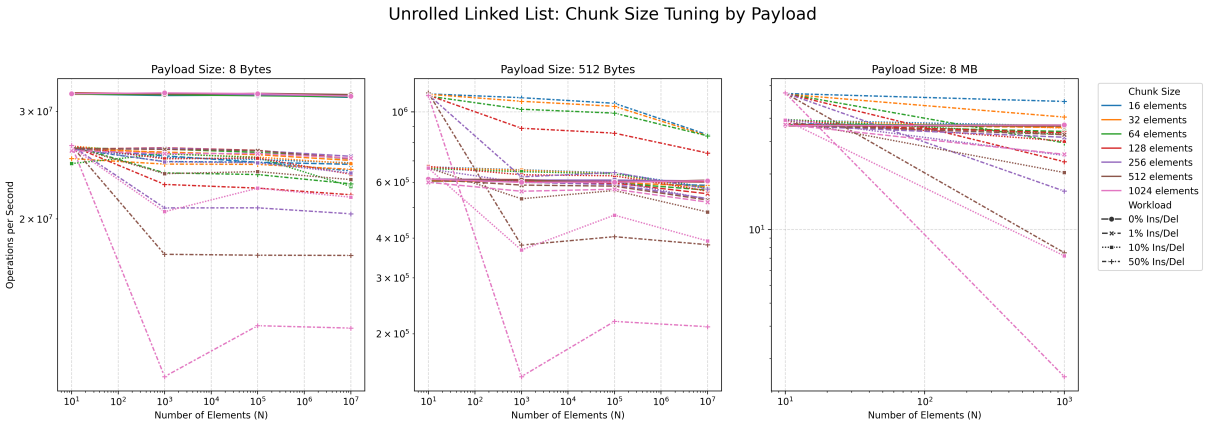
1 Unrolled Linked Lists **1**

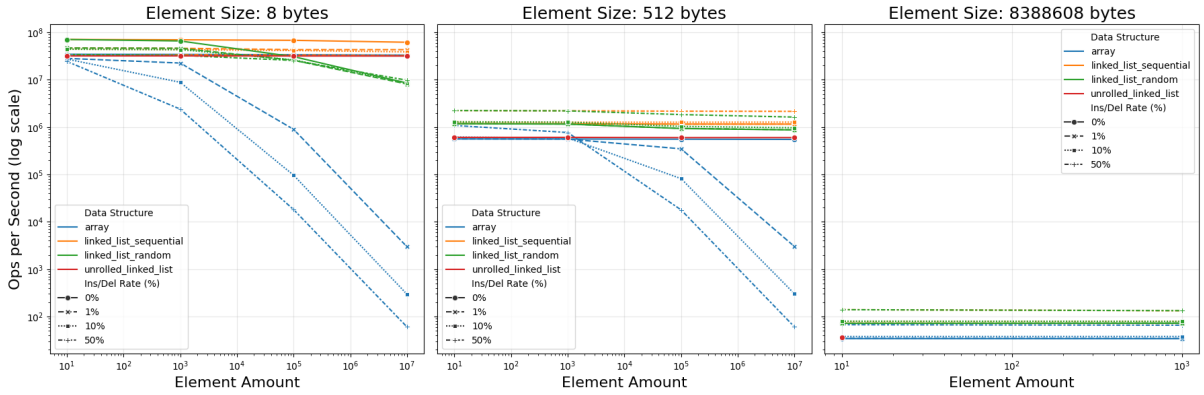
2 Tiered Arrays **2**

3 Extended Benchmarking **4**

1 Unrolled Linked Lists

In this assignment, we compared the performance of arrays against linked lists under different workloads. The baseline Array was implemented using `std::vector<std::vector<char>>`. By structuring the array in this way, the $O(N)$ penalty of copying massive payloads during heavy insertions is avoided, as only the small vector headers need to be shifted in memory. For the Unrolled Linked List, a `std::list<std::vector<char>>` was used. This structure provides the L1 cache locality of vectors, while allowing the linked list to handle chunk splitting when a chunk exceeds its capacity.



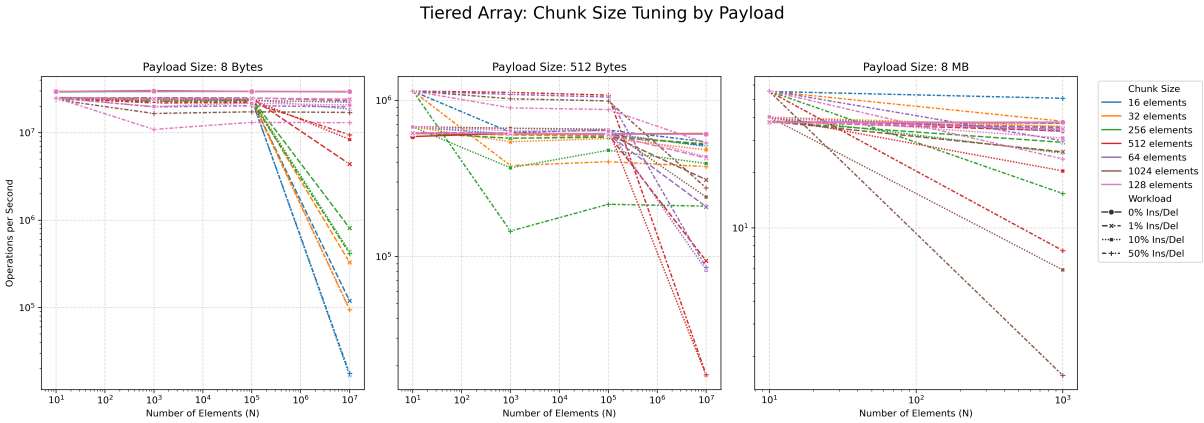


The unrolled list was tuned with different chunk sizes ranging from 16 to 1024 elements. When scaled up to 8 MB elements, the performance of all tested data structures flatlined at roughly 100 operations per second. At this scale, the theoretical difference between $O(1)$ and $O(N)$ seems to become largely irrelevant, likely because the CPU is entirely bottlenecked by the physical memory bandwidth required to process the massive payloads compared to 8 byte.

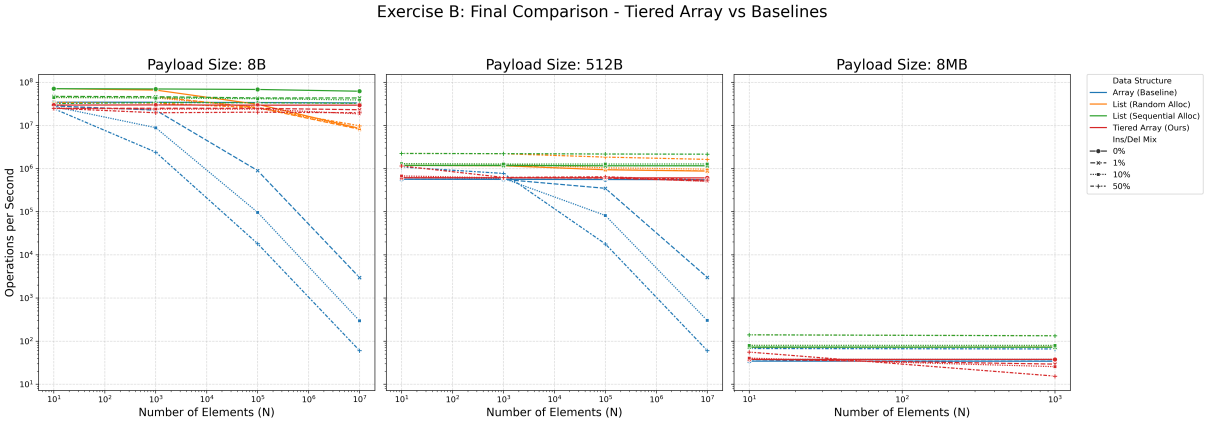
The 50% insert/delete workloads were faster than the 0% pure read/write workloads. Initially, this contradicts the assumption that pure traversal should be faster than shifting memory. Upon reviewing the code of the benchmark, it seems this might be caused by a differences in the computational workload. A read operation is forced to execute a pseudo-random number generator loop for every single byte, meaning a 512-byte read requires 512 separate math iterations. While, insertions and deletions rely on standard vector modifications. It is likely that these modifications use highly optimized, hardware-level SIMD instructions to shift memory efficiently within the CPU cache. Shifting 512 bytes in hardware is most likely faster than calculating 512 individual math equations, suggesting that a heavy computational workload can hide the expected structural efficiencies of a data container.

2 Tiered Arrays

The Tiered Array was implemented as a two-layer structure using a `std::vector` of `std::vector<char>` chunks. Similar to the baseline described in Section 1, this approach avoids large memory copies during insertions by limiting shifting operations to a single chunk. When a chunk exceeds its maximum capacity it is split into two halves and a new entry is added to the outer index vector.



The data structure was tuned using chunk sizes between 8 and 2048 elements to find the most efficient configuration. The results show that a chunk size of 256 elements offers the best performance balance. Smaller sizes increase the overhead of the outer indirection layer while larger sizes lead to performance degradation during insertions as the chunks begin to behave like standard arrays.



In the final comparison the Tiered Array shows a performance trade-off. For pure sequential access it is slower than the sequentially allocated linked list because the two-layer indirection and chunk boundary checks introduce more overhead than simple pointer increments. The cause of the other data structures outperforming the Tiered Array could also be the same as stated in exercise A. However it outperforms the randomly allocated linked list when using more elements at 8 byte. Furthermore while the performance of the baseline Array collapses to near zero during 50% insertion workloads at large N the Tiered Array maintains a high and stable throughput. At the 8 MB payload scale the performance is again limited by memory bandwidth and all structures converge to the same result.

3 Extended Benchmarking

The benchmark was extended to include random access patterns. To simplify implementation, for each data structure an adapter was added, which specified the needed function calls for each data structure. The patterns were pre-generated before the benchmark to not impact performance.

The general benchamark settings were:

1. Maximim RAM usage of 8GB
2. The chunk/batch size for running operations: 256
3. Chunk size for data structures from a) and b)
4. $2s$ warmup time per benchmark
5. $5s$ runtime per benchmark

For the local benchmark, the following machine was used:

1. AMD Ryzen 5 7600X
   (a) 6 Cores
   (b) L1 384 KB
   (c) L2 6 MB
   (d) L3 32 MB
2. 32GB DDR5 6000 mhz

When looking at the plots, one can observe the expected performance impact of random access patterns. Across all tested data structures, random access generally results in lower throughput than sequential access. This behaviour is to be expected because of CPU memory hierarchies. Sequential benefits from cache locality and hardware prefetching, but random access causes more cache misses and/or stalls.
There is one exception however because arrays with very large numbers of elements, see better performance with random access. A possible explanation lies in the benchmark implementation. Under sequential access patterns, these shifts may affect a larger portion of the array on average, and this overhead could be the reason for worse performance than the randomly accessed array.
For all other data structures, the performance gap between sequential and random access becomes increasingly pronounced as the number of stored elements grows. This is also to be expected because with growing number of elements, the chance of cache misses increases even more.
In contrast, increasing the element size has a comparatively small to no influence on the relative difference between sequential and random access.
Another observation is that data structures with contiguous memory layouts, meaning the array and the tiered arrays, benefit more from sequential access than linked-list-based structures. This is again to be expected because contiguous storage allows the processor to exploit spatial locality and cache prefetching more effectively.

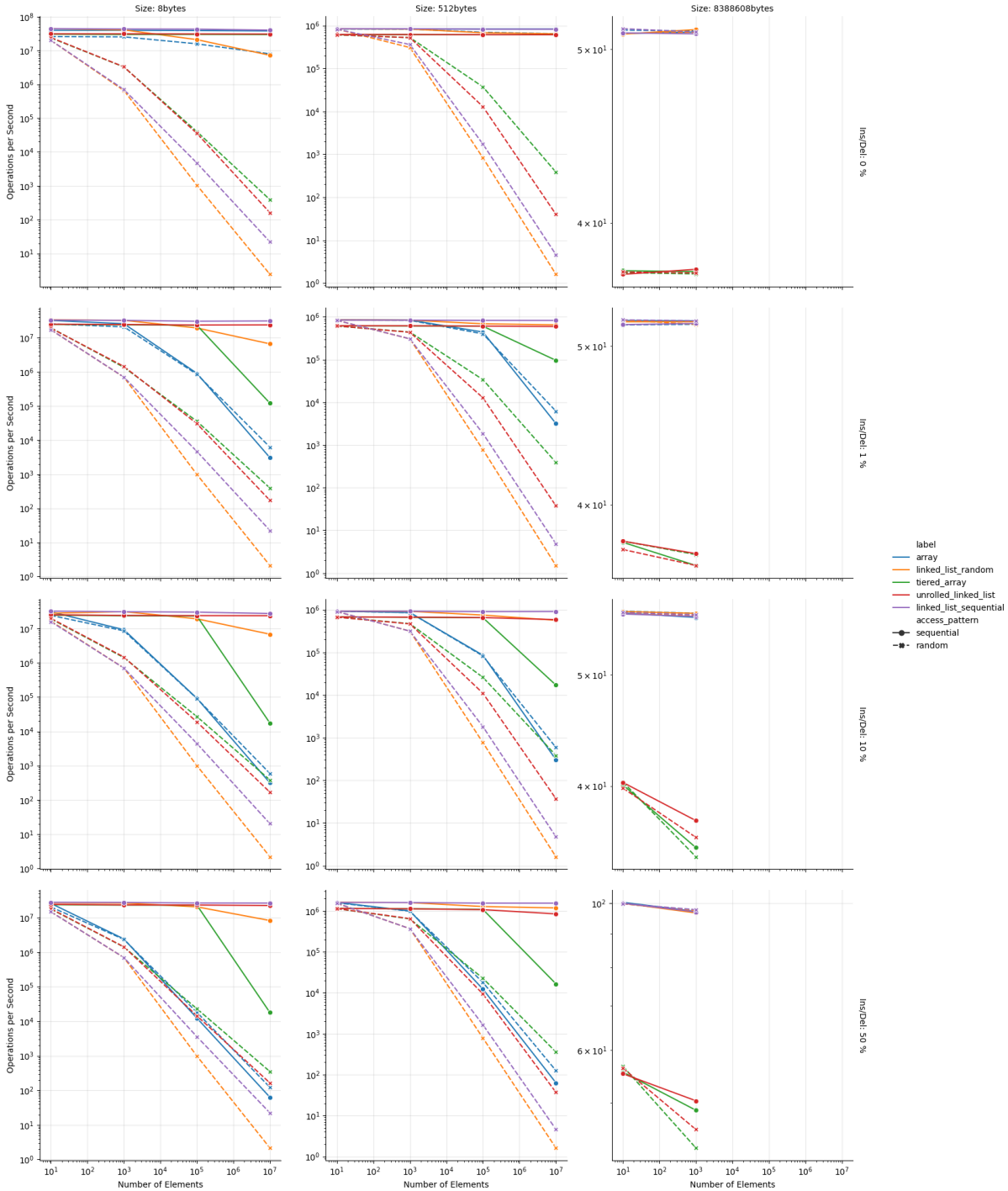


Figure 1: Benchmark Results LCC3

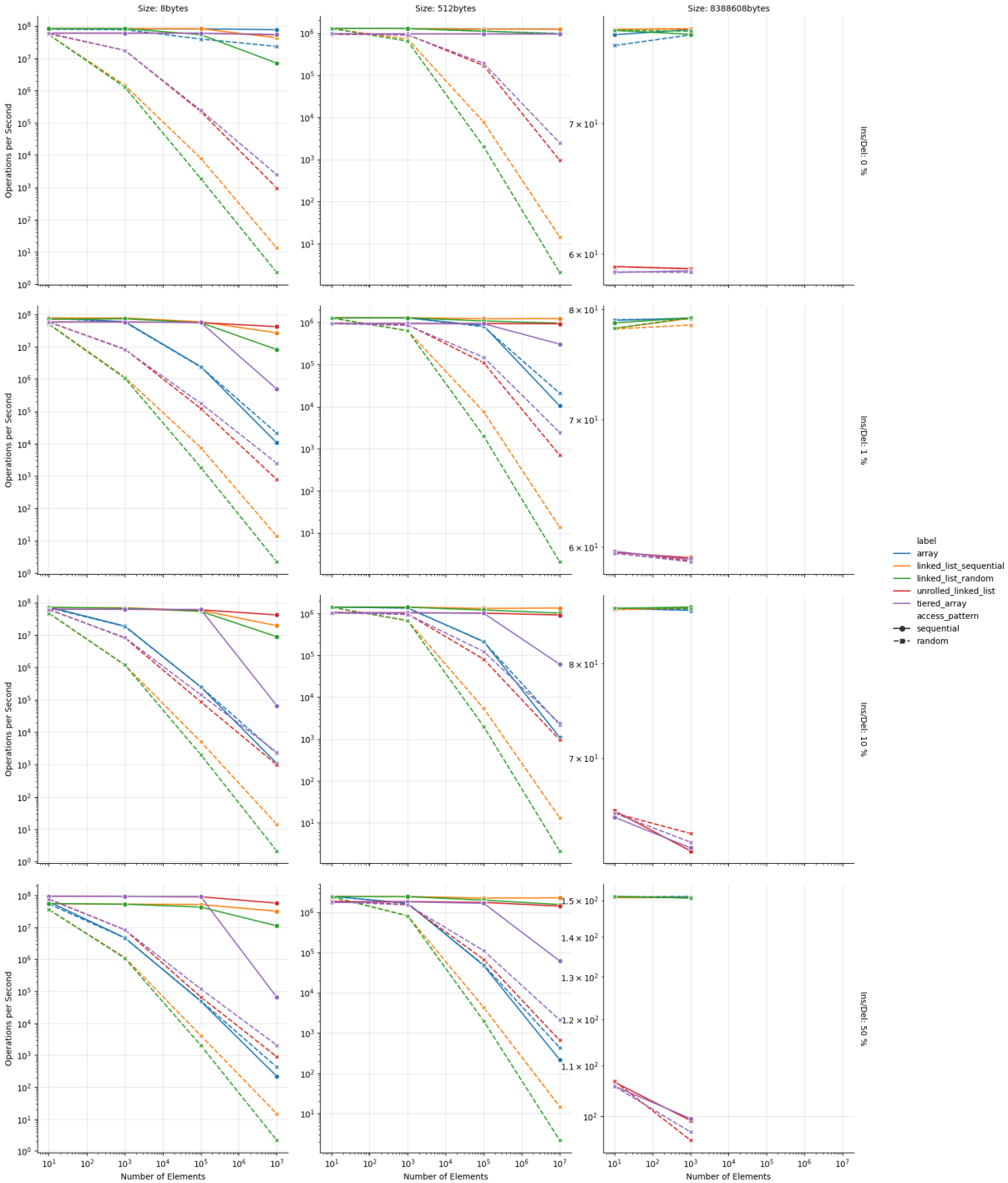


Figure 2: Benchmark Results Local